Monte Carlo simulation of magnetic nanodots on graphics hardware: a single fused kernel for the extended Heisenberg chain

Abstract

This section carries the simulator that produces the spin configurations used throughout the remainder of the work. It states the lattice model and the multi-sublattice Metropolis chain it borrows, then defines the proposal: the realisation of one pass of that chain as a single fused kernel, built from four components named for the properties they deliver — contiguous neighbour access, a synchronisation-free halo, counter-based proposals and deterministic observables. The simulator is validated against a reference implementation of the same chain under identical randomness, and its rate is reported per single-site update, the unit this literature established, together with the per-site arithmetic weight of each model so that the comparison can be read for what it is. At the production workload — 200 temperatures by 20 000 sweeps for one hundred states at five thousand replicas — it completes in 57.6 min on four datacentre devices against 130.7 d on the processor baseline, a separation of 834-fold per site update on identical physics.

1 Materials and methods

This section carries the machinery the simulator borrows and then the simulator itself. Two subsections of existing theory — the lattice model and the checkerboard Metropolis chain — are followed by the proposal, which is the single-kernel realisation of that chain.

1.1 The lattice model

Let $\Lambda \subset \mathbb{Z}^3$ be the set of lattice sites of a cylindrical nanodot,

$$\Lambda = \{(x, y, z) \in \mathbb{Z}^3 : x^2 + y^2 \le R_d^2,\ 0 \le z < L\}, \tag{1}$$

where R_d is the dot radius in lattice steps and L the number of layers. Writing $L_2 = \lfloor R_d \rfloor + 1$ and $N = 2L_2 + 1$, the sites are addressed inside a box of $L \times N \times N$, of which $|\Lambda|$ lie inside the disc. Because N is odd by construction, the box extent carries no factor of two — a fact the proposal uses in Section 1.3.

To each site $i \in \Lambda$ is attached a classical spin $\mathbf{s}_i \in \mathbb{R}^3$ with $\|\mathbf{s}_i\| = S_0$; the configuration is $\mathbf{s} = \{\mathbf{s}_i\}_{i\in\Lambda}$. Let $\mathcal{N}(i) = \{i + \mathbf{d} : \mathbf{d} \in \mathcal{D}_k,\ k \in \mathcal{S}\} \cap \Lambda$ be the interacting neighbourhood of i, and let $\hat{\mathbf{x}}, \hat{\mathbf{y}}, \hat{\mathbf{z}}$ be the lattice directions. The in-plane directions wrap; z does not, so the top and bottom layers have no neighbour beyond them.

The energy has five terms:

$$\begin{aligned} \mathcal{H}(\mathbf{s}) = &- \sum_{k\in\mathcal{S}} J_k \sum_{\langle ij\rangle_k} \mathbf{s}_i \cdot \mathbf{s}_j \ + \ \sum_i \mathcal{A}_i(\mathbf{s}_i) \\ &- \ D \sum_i \Big(\big[\mathbf{s}_i \times \mathbf{s}_{i+\hat{\mathbf{x}}}\big]_y - \big[\mathbf{s}_i \times \mathbf{s}_{i+\hat{\mathbf{y}}}\big]_x \Big) \ - \ \mathbf{H} \cdot \sum_i \mathbf{s}_i . \end{aligned} \tag{2}$$

The first term is the exchange over the active shells $\mathcal{S} \subseteq \{1, 2, 3, 4\}$, where $\langle ij\rangle_k$ runs once over each bond of shell k and $\mathcal{D}_k$ is that shell's displacement set: the six displacements at squared

distance 1, the twelve at 2, the eight at 3 and the six at 4. The first-shell constant is normalised, $J_1 = 1$, and sets the energy scale. The Dzyaloshinskii–Moriya term is *interfacial*: it runs over in-plane bonds only, with the sign shown. That restriction is a modelling commitment, not a simplification — the all-axes form is a different physical system, and on a common configuration the two give total energies that differ by tens of percent.

The anisotropy $\mathcal{A}_i$ is the cubic series written on direction cosines, with a site-dependent first constant:

$$\mathcal{A}_i(\mathbf{s}) = \frac{K_i^{\mathrm{eff}}}{S_0^4}\left[(s_x s_y)^2 + (s_x s_z)^2 + (s_y s_z)^2\right] + \frac{K_2}{S_0^6}(s_x s_y s_z)^2, \qquad K_i^{\mathrm{eff}} = \begin{cases} K_S & i \in \partial\Lambda, \\ K_1 & \text{otherwise}, \end{cases} \tag{3}$$

where $\partial\Lambda$ is the set of sites with fewer than six in-disc first neighbours. Surface anisotropy therefore enters as a *field over sites*, not as a scalar, and that is why it is carried as an array of the same shape as one spin component throughout.

1.2 The multi-sublattice Metropolis chain

Let $\beta = 1/k_B T$. A single-site Metropolis step at site i draws a trial spin $\mathbf{t}_i$ uniformly on the sphere of radius S_0, forms $\Delta\mathbf{s}_i = \mathbf{t}_i - \mathbf{s}_i$, and accepts with probability $\min\{1, e^{-\beta\Delta E_i}\}$, where ΔE_i is the energy difference the single-site change produces. Because every term of (2) couples i only to $\mathcal{N}(i)$ and to itself, ΔE_i reads

$$\Delta E_i = -\,\Delta\mathbf{s}_i \cdot \sum_{k\in\mathcal{S}} J_k \sum_{\mathbf{d}\in\mathcal{D}_k} \mathbf{s}_{i+\mathbf{d}} + \mathcal{A}_i(\mathbf{t}_i) - \mathcal{A}_i(\mathbf{s}_i) - D\,\delta_i^{\mathrm{DM}} - \mathbf{H}\cdot\Delta\mathbf{s}_i, \tag{4}$$

with δ_i^{DM} the in-plane cross-product difference of the DMI term. The DMI term stays on first-shell bonds whatever else is active, because it is interfacial by construction. The quantity that matters for what follows is the *weighted neighbour sum*

$$\mathbf{v}_i = \sum_{k\in\mathcal{S}} J_k \sum_{\mathbf{d}\in\mathcal{D}_k} \mathbf{s}_{i+\mathbf{d}}; \tag{5}$$

it is the only part of ΔE_i that reaches outside site i.

Let $\gamma : \Lambda \to \{0, \ldots, n_c - 1\}$ be a colouring of the lattice, with colour classes $\Lambda^{(c)} = \gamma^{-1}(c)$ so that $\Lambda = \bigsqcup_c \Lambda^{(c)}$. The colouring is *admissible* for the active shells when no displacement of any active shell preserves it,

$$\gamma(i) \neq \gamma(i + \mathbf{d}) \qquad \text{for every } \mathbf{d} \in \mathcal{D}_k,\ k \in \mathcal{S}. \tag{6}$$

Under an admissible colouring, ΔE_i for $i \in \Lambda^{(c)}$ depends on no spin of $\Lambda^{(c)}$, so the sites of one colour may be updated *simultaneously* without changing the chain: the simultaneous update is an exact reordering of the sequential sweep, not an approximation of it. One sweep is n_c passes, $c = 0$ through $n_c - 1$.

Admissibility is decided by the parity of each shell's displacement sums. Shells 1 and 3 have odd sums, so every such neighbour falls in the opposite class of a two-colouring and the interaction graph stays bipartite. Shells 2 and 4 have even sums and it does not. Shell 4 further reaches distance two along a single axis, which no stride-two colouring separates, and therefore requires stride three on all three axes. The narrowest admissible colouring for each case is given in Table 1.

The colouring in use is *verified* against (6) over the displacement sets of the active shells rather than taken from Table 1, so a combination the table does not anticipate is detected instead of assumed.

Table 1: The narrowest admissible colouring for each combination of active shells. The cost of a wider colouring falls on the parallelism and not on the work: a sweep performs $|\Lambda|$ site updates whatever n_c is, but each pass performs only $|\Lambda|/n_c$ of them concurrently.

active shells $\mathcal{S}$	$\gamma(x, y, z)$	n_c
$\{1\}$ or $\{1, 3\}$	$(x + y + z) \bmod 2$	2
any set containing 2, none containing 4	$(x \bmod 2,\ y \bmod 2,\ z \bmod 2)$	8
any set containing 4	$(x \bmod 3,\ y \bmod 3,\ z \bmod 3)$	27

1.3 The proposal: a single fused kernel for the sweep

The chain of Section 1.2 is the object being realised, and the proposal is the realisation of one of its passes as a single fused kernel. It is defined by four components, each named for the property it delivers, and it computes the weighted neighbour sum $\mathbf{v}_i$ of (5) without ever materialising a translate of the configuration.

1.3.1 Contiguous neighbour access: the component-planar layout

Let B be the number of independent replicas advanced together. The configuration tensor may be laid out component-minor, with shape $(B, L, N, N, 3)$, or *component-planar*, with shape $(B, 3, L, N, N)$. The proposal takes the second. Under it, a run along x at fixed (b, a, z, y) — replica, component, layer, row — is contiguous in memory with unit stride, so a row of N spin components of one Cartesian component is a single contiguous segment. The transposition between layouts is performed once, outside the sweep loop, and therefore does not enter the per-sweep cost.

This component is a precondition for the next one: a shifted read is contiguous only if the unshifted read was.

1.3.2 A synchronisation-free halo: the shifted-load stencil

Let a thread block own a tile of B_y rows by B_x lanes of one layer. The stencil requires, for each of the three components, the tile itself and its four in-plane translates together with the two out-of-plane ones. The proposal obtains each translate as a *vector-shifted contiguous read*: the index expression is $x + 1$ applied to a contiguous run, so the access remains a contiguous segment displaced by one element and its contiguity is statically evident. The arithmetic of (5) is then carried out in registers and only the accepted configuration is written.

The halo requires *no barrier*. By the admissibility condition (6), every neighbour a block reads belongs to a colour class other than the one being updated, and is therefore written by no block during that pass. A block may read its halo from global memory with no coordination whatever. The colouring, introduced in Section 1.2 as a correctness device, is thus also what makes the locality safe — and the guarantee holds for any n_c, so it extends to the eight- and twenty-seven-class colourings unchanged.

1.3.3 Counter-based proposals: in-kernel generation

The trial spin and the acceptance variate are generated *inside* the kernel from a counter-based generator rather than supplied by the host. For site i of replica b at step n, let $\omega(b, i) = ((bL + z_i)N + y_i)N + x_i$ be the global site offset. The proposal draws

$$g_k = \mathcal{N}(n,\ 4\omega + k),\ k \in \{0, 1, 2\}, \qquad u = \mathcal{U}(n,\ 4\omega + 3), \qquad \mathbf{t}_i = S_0 \frac{(g_0, g_1, g_2)}{\|(g_0, g_1, g_2)\|}, \tag{7}$$

with $\mathcal{N}$ and $\mathcal{U}$ the normal and uniform streams of the generator. The four draws share one key, the step index n, and are separated by *distinct counters* — the standard use of a counter-based generator. The separation is required rather than incidental: the acceptance variate u and the direction $\mathbf{t}_i$ must be statistically independent for the acceptance test of Section 1.2 to sample the intended chain, and distinct counters is what guarantees they share no input.

1.3.4 Deterministic observables: block-partial reduction

The observables of (8) below are accumulated as *per-block partial sums*, written to a small array and reduced afterwards. The reduction is therefore independent of the order in which blocks retire, so two evaluations of the same configuration agree to the last bit and the observables admit exact comparison. The partial array is of size $O(BL\lceil N/B_y\rceil)$ and its final reduction is negligible beside the sweep.

The topological charge is the one observable whose kernel is partial by construction: the Berg–Lüscher plaquette form requires a two-argument arctangent whose branch matters, since its denominator may change sign on a frustrated plaquette. The kernel emits the four arguments of the two arctangents and the final transcendental is applied outside it.

1.3.5 The observables

Let $n_a = |\Lambda|$. For a configuration $\mathbf{s}$, the simulator reports the energy per site, the magnetisation and its out-of-plane component, and the topological charge of the mid-layer:

$$E = \frac{1}{n_a}\,\mathcal{H}(\mathbf{s}), \qquad \mathbf{M} = \frac{1}{n_a}\sum_{i\in\Lambda}\mathbf{s}_i, \qquad Q = \frac{1}{4\pi}\sum_{p}\Omega_p, \tag{8}$$

where the energy is evaluated with each bond counted once, using only the $+\hat{\mathbf{x}}$, $+\hat{\mathbf{y}}$ and $+\hat{\mathbf{z}}$ translates, and Ω_p is the signed solid angle of plaquette p,

$$\Omega(\mathbf{a},\mathbf{b},\mathbf{c}) = 2\arctan\frac{\mathbf{a}\cdot(\mathbf{b}\times\mathbf{c})}{1+\mathbf{a}\cdot\mathbf{b}+\mathbf{b}\cdot\mathbf{c}+\mathbf{c}\cdot\mathbf{a}}, \tag{9}$$

each plaquette contributing $\Omega(\mathbf{s}_{00},\mathbf{s}_{10},\mathbf{s}_{11}) + \Omega(\mathbf{s}_{00},\mathbf{s}_{11},\mathbf{s}_{01})$ over the plaquettes whose four corners lie in the disc. This form is quantised, unlike a finite-difference curl.

The response functions are variances,

$$C_v = \frac{\langle(\Delta E)^2\rangle}{k_B T^2 n_a}, \qquad \chi = \frac{\langle(\Delta\|\mathbf{M}\|)^2\rangle}{k_B T}, \tag{10}$$

and they are accumulated about a reference value taken once per temperature. A constant shift leaves a variance unchanged, so this is exact, and it keeps every accumulated quantity at the scale of the fluctuation rather than of the total — which is what single precision requires here, since at low temperature one unit in the last place of $\langle E^2\rangle$ exceeds the variance being measured.

1.3.6 The combination

One sweep applies the kernel of Sections 1.3.1–1.3.3 once per colour class, n_c times; one temperature applies the sweep n_{th} times to thermalise and n_{me} times while accumulating the observables of Section 1.3.4; and one simulation scans a decreasing temperature ladder $\{T_m\}_{m=1}^{M}$, carrying the final configuration of each temperature into the next:

$$\mathbf{s}^{(m)} = \Big(\bigcirc_{c=n_c-1}^{0}\,\mathcal{K}_{\beta_m}^{(c)}\Big)^{\,n_{\mathrm{th}}+n_{\mathrm{me}}}\!\big(\mathbf{s}^{(m-1)}\big), \qquad \beta_m = 1/k_B T_m, \tag{11}$$

where $\bigcirc$ denotes composition over the colour classes in descending order, $\mathcal{K}_\beta^{(c)}$ is the single-pass kernel for class c at inverse temperature β, and the whole composition is evaluated for the B replicas at once. Every symbol traces to a declaration: $\mathcal{K}^{(c)}$ to Section 1.3.2, the classes c and their count n_c to (6), β to Section 1.2, and B to Section 1.3.1.

Figure 1 summarises the four components in the order this section defines them, showing for one pass how the layout, the halo, the generator and the reduction compose into the kernel.

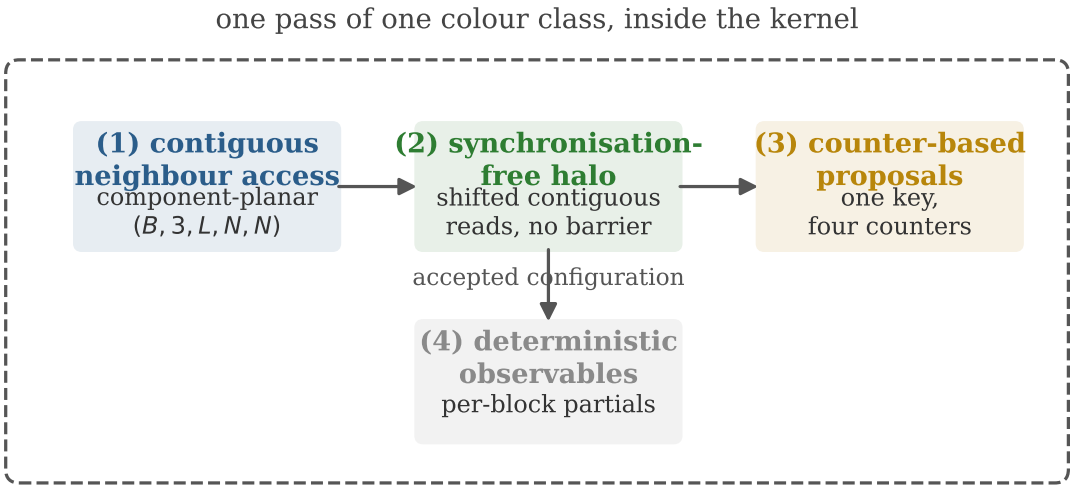


Figure 1: The fused single-pass kernel. (1) *Contiguous neighbour access*: the configuration is held component-planar, so a row of one Cartesian component is a contiguous segment. (2) *A synchronisation-free halo*: each block reads its tile and the one-site border through shifted contiguous reads, with no barrier, because the halo is of the opposite colour. (3) *Counter-based proposals*: the trial direction and the acceptance variate are drawn in-kernel from one key and four distinct counters. (4) *Deterministic observables*: per-block partial sums are written to a small array and reduced outside the kernel. Colour marks the component a box belongs to; the dashed boundary is the kernel.

2 Position in the literature

A full review of accelerated lattice Monte Carlo belongs to its own section; what follows is the part of it needed to read the rate reported in Section 3 for what it is.

Accelerating lattice spin Monte Carlo on graphics hardware has two largely separate literatures, and the simulator of Section 1.3 sits between them. One optimises the cheapest possible site update to its hardware limit; the other carries a physically rich Hamiltonian and accepts whatever rate follows. Both rest on the same structural observation, which predates any particular implementation: a bipartite colouring of the interaction graph makes one colour's updates mutually independent, so a sweep becomes two data-parallel passes. The differences lie in what each does with the freedom that buys.

The first line compresses the state. If a spin takes two values it fits in one bit, and a machine word then holds dozens of independent lattice sites or dozens of independent replicas, updated by bitwise operations in a single instruction. Multi-spin coded implementations of the two-dimensional Ising model established both the method and the benchmark unit still in use, reporting a single-device rate on 2010 hardware and scaling across sixty-four devices to aggregate rates two orders of magnitude above one device [1]. Subsequent work sharpened the packing rather than the parallelism, and the most recent entries move to rack-scale systems and report aggregate lattice-update rates another two orders of magnitude higher [7]. A parallel strand pursues the same compression for spin glasses, where asynchronous multi-spin coding

across replicas serves parallel tempering rather than a single long chain [4]; the structural lesson there transfers directly — when the physics requires many weakly coupled chains, the replica axis is a better place to find parallelism than the lattice, because it needs no halo at all.

Two properties of that line matter here. The rates are genuinely remarkable and they are also *model-specific*: the bit packing that produces them requires a spin with two states, and a continuous spin admits no analogue. And the quantities those implementations compute are correspondingly narrow — magnetisation and energy of a two-state model, with no anisotropy, no chiral interaction and no vector observable.

The second line keeps the vector spin. Weigel's surveys of lattice spin models on graphics hardware cover both the Metropolis update and cluster algorithms and report per-update rates for continuous models on hardware of their period [2, 3], obtained in part by running many independent systems concurrently — the same replica-axis strategy, arrived at independently. The closest published work to the simulator of Section 1.3 is Charilaou's Monte Carlo implementation of the three-dimensional anisotropic Heisenberg model [6], which carries continuous spins and an anisotropy and reports a per-update time together with a demonstration at one hundred million spins on a single device. It is the only entry whose Hamiltonian is of the same kind as the one in Section 1.1, and therefore the only one against which a per-update comparison carries meaning. Beyond Monte Carlo, the atomistic spin dynamics community maintains frameworks that integrate the Landau–Lifshitz–Gilbert equation on the same kind of lattice and with a comparable interaction set [5]; their figures are reported in iterations per second rather than per-update times, which makes them the natural comparison for a *production workflow* and the wrong comparison for a *sweep rate*.

The reported rates across these works differ by four orders of magnitude, and the gap is almost entirely the choice above rather than engineering quality. That makes the literature difficult to read as a ranking, and reading it as one is a recurring error: the unit shared across these works, time per single-site update, is not commensurable across models whose per-site arithmetic differs by two orders of magnitude. Table 2 records which properties each line supplies.

Table 2: Synthesis of the reviewed lines against the three properties this work requires. A dash marks a property the source does not report rather than one it lacks.

line of work	continuous spin	chiral + surface anisotropy	per-update rate reported	multiple architectures
bit-compressed Ising [1, 7]	no	no	yes	—
Ising spin glass, multi-spin [4]	no	no	yes	—
continuous-spin surveys [2, 3]	yes	no	yes	—
anisotropic Heisenberg MC [6]	yes	partial	yes	—
atomistic spin dynamics [5]	yes	yes	no	partial
this work	**yes**	**yes**	**yes**	**yes (2)**

No reviewed line fills every column. The implementations that report the highest rates carry the least physics, and the one whose Hamiltonian most resembles the model of Section 1.1 reports a rate an order of magnitude slower than the bit-packed codes of a decade earlier. No reviewed work carries an interfacial chiral interaction, a site-dependent surface anisotropy and a cubic anisotropy series together, and none reports a per-update rate for such a model. That combination is the gap the kernel of Section 1.3 was built for.

A fourth property is methodological rather than computational, and it recurs across both literatures: performance claims are generally reported without a stated control against the implementation being replaced, and where a reimplementation is compared against a reference, the comparison is usually of final observables rather than of the chain itself — which cannot distinguish an engine that samples the right distribution from one whose sampling is subtly biased but whose equilibrium averages happen to agree. The validation protocol of Section 3

addresses that directly.

3 Results and discussion

3.1 The simulator samples the intended chain

Table 3 collects the four exactness tests and the three statistical ones, with the number each returns. Fed identical randomness, the fused kernel of Section 1.3.2 and the reference implementation of the chain agree on every spin of the configuration (0 of 182 520 differing, Table 3 row 1), with the single-site energy difference matching to 1.4×10^{-6} in absolute value, which is the resolution of single precision on that quantity. The observable kernels of Section 1.3.4 agree with the reference observables on a common configuration to 1.2×10^{-7} relative for the energy and 8.4×10^{-8} absolute for the topological charge (rows 3 and 4). These rows carry no statistics: with the randomness held fixed the only admissible difference is arithmetic, so a non-zero spin count would identify a term of (2) wrongly transcribed rather than a sampling discrepancy.

Table 3: Agreement with the reference implementation of the chain. Rows 1–4 hold the randomness fixed and are exactness tests; rows 5–7 let each engine draw its own and are statistical.

#	test	quantity	result
1	one pass, even class, identical randomness	differing spins	0/182 520
2	one pass, odd class, identical randomness	differing spins	0/182 520
3	observable kernels, energy	relative	1.2×10^{-7}
4	observable kernels, topological charge	absolute	8.4×10^{-8}
5	both chains to equilibrium, $T = 20, 5, 1$	σ	$0.09 - 0.23$
6	trial direction norm	deviation from S_0	2.4×10^{-7}
7	reduced curves, reference run against itself	mean σ / max σ	0.99/1.90

Row 7 is the instrument that settles the question, and reading it requires the control the row names. Running the reference implementation against *itself* with a second seed returns 0.87 and 2.53 for the same two statistics, which is the noise floor of the comparison rather than a property of either engine; the fused kernel's 0.99 and 1.90 therefore sit inside that floor, its maximum deviation falling below the control's own. Without that control a figure in this row is not interpretable in either direction, because the two engines draw from different generators and are therefore distinct Markov chains that can agree only in distribution.

The admissibility condition (6) is checked computationally for the colouring in use rather than argued from Table 1. Over the displacement sets of the active shells, the two-class colouring is admissible for shells $\{1\}$ and $\{1, 3\}$, the eight-class colouring for any set containing shell 2, and the twenty-seven-class colouring for any set containing shell 4; in every case the colour classes partition the sites inside the disc exactly. Because the no-barrier guarantee of Section 1.3.2 follows from that condition and not from the class count, it holds unchanged at eight and twenty-seven classes.

3.2 Each component delivers the property it was defined under

3.2.1 Contiguous neighbour access and the synchronisation-free halo

Table 5 reports the per-site update time the two components jointly achieve: 0.129 ns on one datacentre GPU (Table 5, row 1), and 0.033 ns aggregated over four, which that table deliberately omits because it tabulates per-device rates only. At the geometry of Section 1.1 that is 7.74 site updates per nanosecond per device. The mechanism is the one the layout was defined for: the weighted neighbour sum (5) is formed from contiguous segments displaced

by one element, so each translate costs one coalesced transaction per segment rather than a gather, and no translate is ever written to memory.

3.2.2 Counter-based proposals

Table 4 presents the acceptance rate of the in-kernel generator against the reference generator at five temperatures spanning the ladder. The ratio scatters on both sides of unity (0.996 to 1.029, Table 4 column 4), each value within one standard error of it, where the standard error of a ratio of two independently measured acceptance rates is $\sqrt{2}\,\sqrt{(1-p)/(Np)}$ and runs from 0.47 % at the hottest temperature to 3.4 % at the coldest, the acceptance rate p falling by two orders of magnitude across that range. The absence of a systematic sign **confirms** the independence that Section 1.3.3 requires between the acceptance variate and the trial direction: a shared input between them would bias the test in one direction at every temperature, which is the signature this instrument is built to detect.

Table 4: Acceptance rate of the in-kernel generator against the reference generator, from a common configuration. The expected ratio is unity; the standard error grows as the acceptance rate falls.

T (K)	reference	in-kernel	ratio	std. error (%)
20.0	0.4233	0.4218	0.996	0.47
10.0	0.1614	0.1611	0.998	0.97
5.0	0.0723	0.0718	0.994	1.50
1.0	0.0132	0.0136	1.029	3.40
0.1	0.0029	0.0029	0.997	7.30

3.2.3 Deterministic observables

Rows 3 and 4 of Table 3 are also this component's evidence, and they are available *as* exactness tests only because the reduction is order-independent: a reduction whose result depended on the order in which blocks retired could be compared only within a tolerance, and the figures of 1.2×10^{-7} and 8.4×10^{-8} would carry no information about the formulation. That the comparison is admissible at all is **attributed to** the partial-sum reduction of Section 1.3.4 rather than inferred from the agreement.

3.3 The simulator at production scale

Figure 2 is the principal result. It presents the wall clock of the complete production ladder — 200 temperatures by 20 000 sweeps, so 4×10^6 sweeps per replica — for one hundred states at five thousand replicas, across the CPU baseline, one and two consumer GPUs and one, two and four datacentre GPUs.

Read at four datacentre GPUs (Figure 2, rightmost bar), the simulator completes the hundred-state ladder in 57.6 min; at one device (Figure 2, fourth bar) in 3.8 h; and on two consumer GPUs (Figure 2, third bar) in 25.8 h. Against the CPU baseline on the identical workload, 130.7 d, the separation is 834-fold per site update (Table 5, rows 1 and 3). Figure 3 shows the same comparison without the CPU bar and on a linear axis.

The hundred-state configuration is the one that exhibits the scaling, which is why it is the principal figure rather than the single-state case. Over one to four devices the simulator gains a factor of 3.91 with a hundred states and only 1.10 with one, on the same hardware and the same kernel. This is **attributed to** the replicas available per device rather than inferred from the two figures: at one state and four devices each device receives twelve replicas, and Figure 4 shows the per-site cost against replicas per device, where the curve rises steeply below roughly

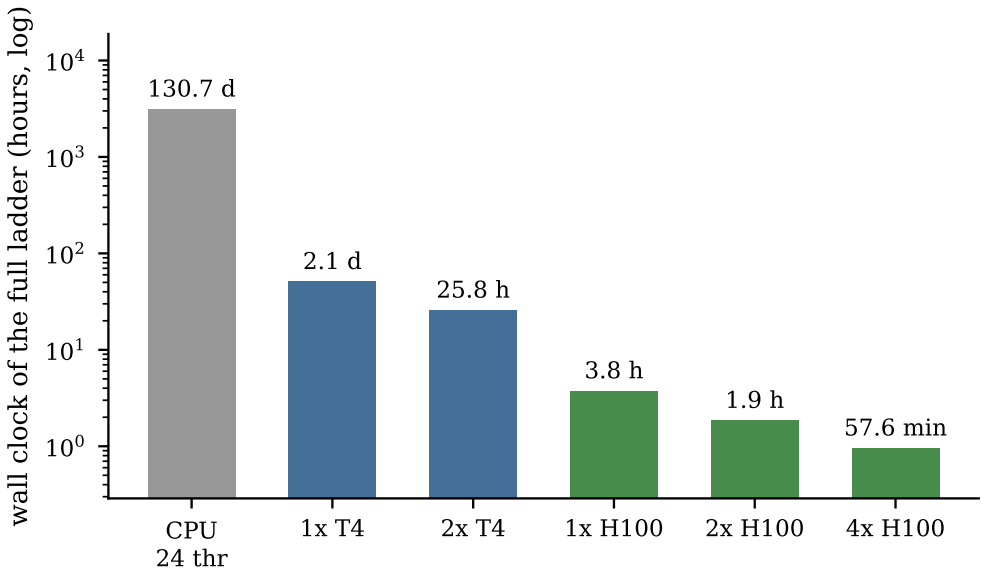


Figure 2: The production ladder for one hundred states at five thousand replicas: 200 temperatures by 20 000 sweeps, 4×10^6 sweeps per replica. Wall clock on a logarithmic axis, since at 130.7 d the processor baseline compresses every other bar onto the axis.

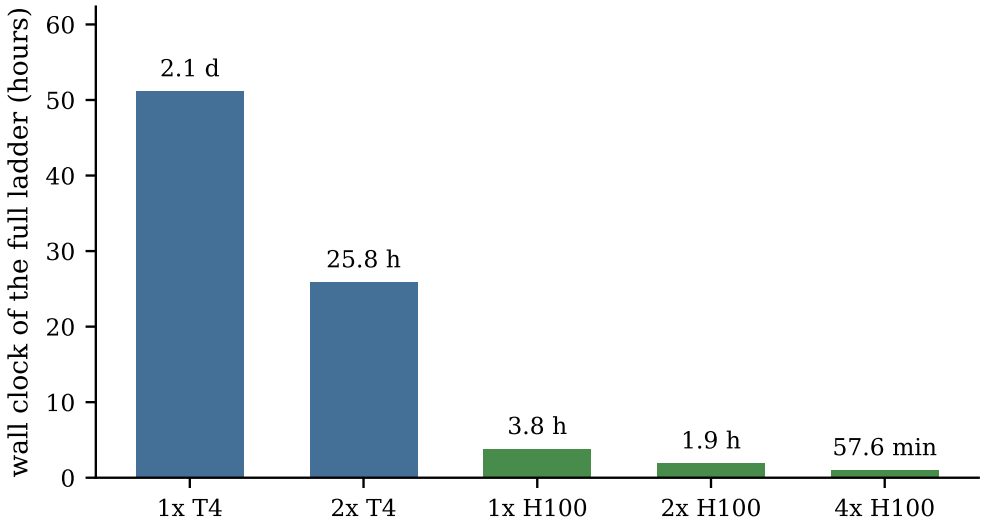


Figure 3: The same ladder as Figure 2 with the processor baseline removed and a linear axis, which is what makes the separation among the accelerated configurations legible.

twenty-five. The practical consequence is that additional devices are paid for by additional states — one state per device requires no sharding and preserves the per-state cost — and not by dividing one state across them.

Figure 5 presents the per-site cost against the replica count at fixed geometry and shows it very nearly insensitive to the arithmetic form of the array extent. Let v_2 be the 2-adic valuation, B the replicas per device and L the layer count. The condition $v_2(B) + v_2(L) \geq 2$ can be flipped by varying B only when $v_2(L) < 2$, which among the layer counts measured holds for $L = 5$ and $L = 6$ and not for $L = 4$ or $L = 8$, where the condition is already satisfied for every B. Across the two informative layer counts the condition changes the cost by 0.30 % and 1.70 % on the datacentre device, and by 0.56 % and 0.97 % on the consumer device — at most 1.7 %, with no consistent sign. This is **attributed to** the formulation: the block shape is fixed by the kernel of Section 1.3.2 and the layer count only sets how many blocks exist along

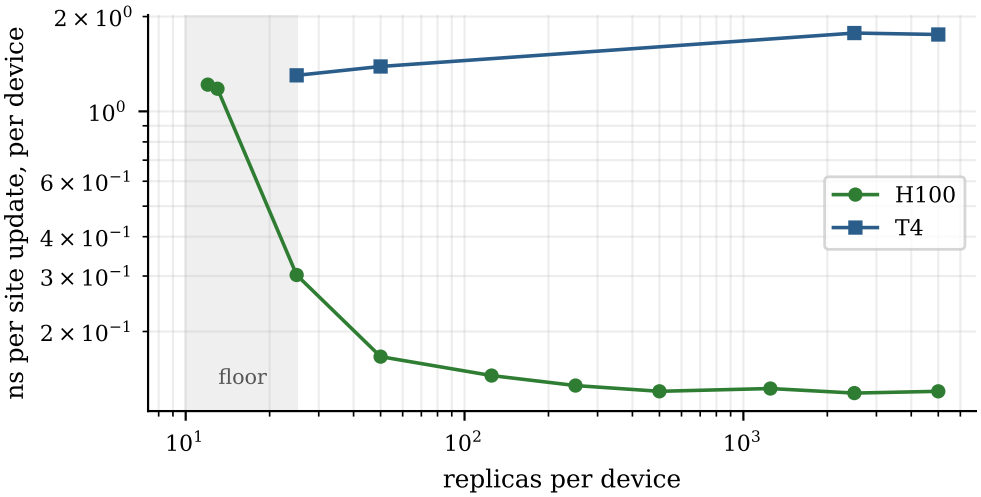


Figure 4: Per-site update cost against replicas per device, at fixed geometry. Below roughly twenty-five replicas per device the curve rises steeply: this is the floor that makes a single state fail to use four devices. The datacentre device improves as the batch widens and then plateaus; the consumer device degrades monotonically over the same range.

the out-of-plane axis, so no code-generation decision depends on the extent. The practical consequence is that the replica count may be chosen for the physics or for the scheduling.

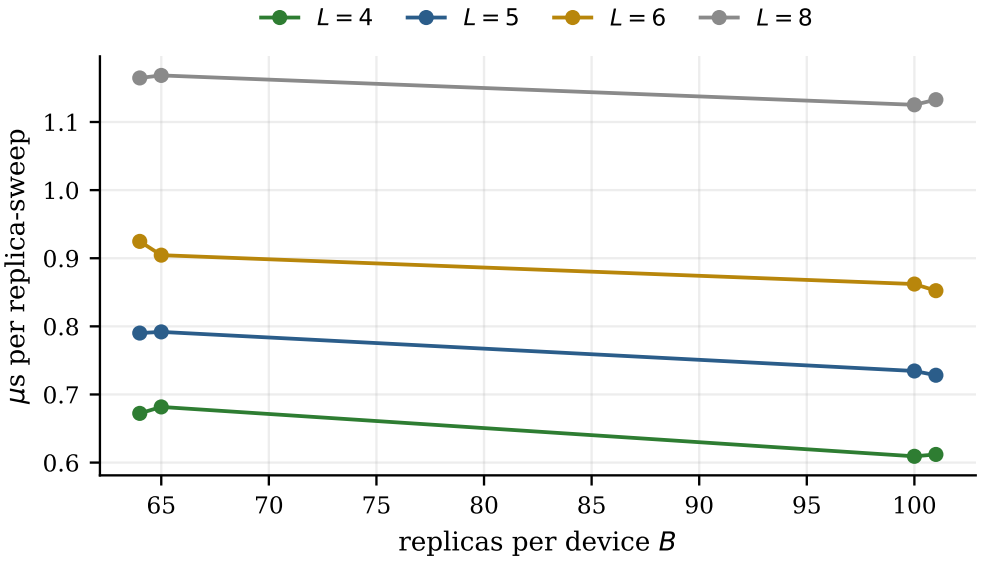


Figure 5: Per-replica-sweep cost against the replica count at fixed geometry, one line per layer count, on the datacentre device. Each pair $B = 64, 65$ and $B = 100, 101$ flips the condition $v_2(B) + v_2(L) \geq 2$ at $L = 5$ and $L = 6$; the lines cross those pairs flat, by 0.30 % and 1.70 %. At $L = 4$ and $L = 8$ the condition holds for every B, so those two lines carry no information about it and are shown for the cost scale.

The margin is not uniform across devices, and Figure 4 shows the sign. On the datacentre device the per-site cost improves as the batch widens and then plateaus at 0.677 µs per replica-sweep; on the consumer device it degrades monotonically, from 7.282 to 9.201 µs over the same range. This **suggests** a memory-hierarchy origin — the consumer device has an order of magnitude less last-level cache and a third of the bandwidth, against a working set of 456 MB at five thousand replicas — but that mechanism was not measured here, and only the sign is established.

3.4 Against published implementations

Table 5 places the simulator beside published GPU Monte Carlo spin codes on a common unit, the time per single-site update.

Table 5: Time per single-site update, per device. This work's rows are all taken at five thousand replicas per device, so they share a basis. Every published figure is read from the paper itself, not from a secondary source, and the rightmost column states the per-site arithmetic each model carries — which is what decides whether a comparison in this unit means anything.

implementation	model	device	ns / update	per-site work
this work	Heisenberg + iDMI + cubic + surface + field	H100	**0.129**	heavy
this work	same	T4	1.75	heavy
this work, CPU baseline	same	24 threads	107.7	heavy
Charilaou [6]	3-D anisotropic Heisenberg	Tesla V100	1.55	moderate
Block *et al.* [1]	2-D Ising, multi-spin coding	Tesla C1060	0.127	minimal
Weigel & Yavors'kii [3]	lattice spin models, $k = 100$	GTX 480	0.03	minimal
Bisson *et al.* [7]	2-D Ising	GB200	5.5×10^{-4}	minimal

The aggregate figures those works report are larger again and are not in the table, because they measure a different thing: 206 updates per nanosecond over sixty-four devices [1] and 1.15×10^5 over a rack [7], against this work's 0.033 ns per update aggregated over four devices. A per-device row is the only one that compares.

Against the published implementation whose Hamiltonian is closest — an anisotropic Heisenberg model, the only entry carrying continuous spins and an anisotropy — the raw ratio is twelvefold (0.129 against 1.55 ns, Table 5 rows 1 and 4). That ratio should not be quoted on its own, because the two rows are measured on hardware two generations apart: the published figure is from a Tesla V100 and this work's from an H100.

The comparison that controls for the device is between row 4 and row 2. This work's consumer-class device returns 1.75 ns against the V100's 1.55, and a V100 carries twice the multiprocessors and nearly three times the memory bandwidth of that device. Normalising on that basis, the formulation is ahead by something between a factor of two and three rather than twelve — smaller than the raw ratio and attributable to the method rather than to the hardware.

The Ising entries are faster per update, by up to two orders of magnitude on a per-device basis, and that is not evidence about this work in either direction. The per-site arithmetic differs by roughly two orders of magnitude: a bit-packed Ising flip is a handful of integer operations with no transcendental and one random number, while one update here performs on the order of 290 arithmetic operations, one exponential, one reciprocal square root and four random draws, as the kernel of Section 1.3.2 makes explicit. Reading Table 5 as a ranking would be reading a unit the models do not share.

What the table supports, read across rows 1 and 5, is narrower and more useful: the simulator reaches the per-device rate of a multi-spin coded Ising implementation on 2010 hardware — 0.129 against 0.127 ns — while carrying a Hamiltonian some two orders of magnitude heavier per site. The comparison that is strictly controlled remains the one against the CPU baseline, where the code, the physics and the workload are identical, and there the figure is 834-fold.

3.5 Limitations

Four limitations bound the claims above. The per-device replica floor, below which the per-site cost rises steeply (Figure 4). The architecture dependence of the margin, whose sign is measured and whose mechanism is not. The widening of the colouring that shells 2 and 4 require, which divides the parallelism available per pass by eight or twenty-seven while leaving the work

per sweep unchanged, and which the fused kernel of Section 1.3.2 does not yet implement beyond the first shell. And the single geometry in which the cost behaviour of Figure 5 was characterised.

References

[1] B. Block, P. Virnau and T. Preis, *Multi-GPU accelerated multi-spin Monte Carlo simulations of the 2D Ising model*, arXiv:1007.3726 (2010).

[2] M. Weigel, *Simulating spin models on GPU*, arXiv:1006.3865 (2010).

[3] M. Weigel and T. Yavors'kii, *GPU accelerated Monte Carlo simulations of lattice spin models*, arXiv:1107.5463 (2011).

[4] Y. Fang, S. Feng, K.-M. Tam, Z. Yun, J. Moreno, J. Ramanujam and M. Jarrell, *Parallel tempering simulation of the three-dimensional Edwards–Anderson model with compact asynchronous multispin coding on GPU*, arXiv:1311.5582 (2013).

[5] G. P. Müller, M. Hoffmann, C. Disselkamp, D. Schürhoff, S. Mavros, M. Sallermann, N. S. Kiselev, H. Jónsson and S. Blügel, *Spirit: multifunctional framework for atomistic spin simulations*, arXiv:1901.11350 (2019).

[6] M. Charilaou, *GPU-accelerated Monte Carlo simulations of anisotropic Heisenberg ferromagnets*, arXiv:2106.11286 (2021).

[7] M. Bisson, M. Bernaschi, M. Fatica, N. G. Fytas, I. González-Adalid Pemartín, V. Martín-Mayor and A. Vasilopoulos, *Massive-scale simulations of 2D Ising and Blume–Capel models on rack-scale multi-GPU systems*, arXiv:2502.18624 (2025).